```
> ##########################################################################
  #######
  # GravitonHfield.mpl
  # ------------------------------------------------------------------------
  -----
  # TauTimp framework: variable-coefficient spherical wave equation
  #
  #   (R/r) Laplacian H(r,t) = (1/c^2) d^2 H / dt^2,  spherically symmetric
  #
  # After separation H = g(r) u(t) with separation constant -k^2:
  #   - u(t) = cos(omega t) + sin(omega t),  omega = k c
  #   - radial:  g'' + (2/r) g' + (k^2 r/R) g = 0
  #   - sub g = h/r  ->  h'' + (k^2 r/R) h = 0   (Airy equation in disguise)
  #   - h(r) = c1 Ai(-alpha r) + c2 Bi(-alpha r),  alpha = (k^2/R)^(1/3)
  #
  # BCs:  g(r0) = 0  and  g(2R) = 0   (r0 = 4.3e-19 m = space quantum)
  # Eigenvalues from det:  Ai(-alpha r0) Bi(-2 alpha R) - Ai(-2 alpha R) Bi
  (-alpha r0) = 0
  #
  # Produces 14 structured plots.
  ##########################################################################
  #######

  restart:
  with(plots):
  with(LinearAlgebra):
  interface(displayprecision = 10):

  ##########################################################################
  #######
  # SECTION 1 : Constants
  ##########################################################################
  #######
  printf("\n============================================================\n");
  printf("  TAUTIMP GRAVITON H-FIELD - SECTION 1: CONSTANTS\n");
  printf("============================================================\n");

  c_light := 2.99792458e8:
  hbar    := 1.054571817e-34:
  R_taut  := 3.9228e22:
  r0_cut  := 4.3e-19:        # space quantum inner cutoff

  printf("R          = %.6e m\n",   R_taut);
  printf("2R         = %.6e m\n", 2*R_taut);
  printf("r0 (space) = %.6e m\n",   r0_cut);
  printf("r0 / R     = %.6e\n",     r0_cut/R_taut);
  printf("c          = %.6e m/s\n", c_light);
  printf("hbar       = %.6e J s\n", hbar);
```

```
#########################################################################
#######
# SECTION 2 : PLOT 1 - Coefficients R/r and 2R/r^2
#########################################################################
#######
printf("\n=== PLOT 1: Variable coefficients R/r and 2R/r^2 ===\n");

p1a := loglogplot(R_taut/r, r = 1e0..2*R_taut,
                  color = "Blue", thickness = 2,
                  legend = "R/r"):
p1b := loglogplot(2*R_taut/r^2, r = 1e0..2*R_taut,
                  color = "Red", thickness = 2,
                  legend = "2R/r^2"):
plot1 := plots:-display([p1a, p1b],
               labels = ["r [m]", "coefficient"],
               title = "Plot 1: PDE coefficients R/r and 2R/r^2 (log-log)",
               gridlines = true):
print(plot1);

#########################################################################
#######
# SECTION 3 : PLOT 2 - Airy functions Ai(xi) and Bi(xi)
#########################################################################
#######
printf("\n=== PLOT 2: Airy functions ===\n");

plot2 := plot([AiryAi(xi), AiryBi(xi)], xi = -10..2,
               color = ["DarkBlue", "DarkRed"], thickness = [2,2],
               legend = ["Ai(xi)", "Bi(xi)"],
               labels = ["xi", "Airy function"],
               title = "Plot 2: Airy functions Ai (blue) and Bi (red)",
               gridlines = true):
print(plot2);

#########################################################################
#######
# SECTION 4 : Eigenvalue computation
#########################################################################
#######
printf("\n=== Computing eigenvalues via fsolve ===\n");

# Determinant function
det_fn := proc(k_val)
    local alpha_v, xi0_v, xi2R_v, Ai_0, Bi_0, Ai_2R, Bi_2R;
    alpha_v := (k_val^2 / R_taut)^(1/3);
    xi0_v   := -alpha_v * r0_cut;
    xi2R_v  := -alpha_v * 2 * R_taut;
    Ai_0    := evalf(AiryAi(xi0_v));
    Bi_0    := evalf(AiryBi(xi0_v));
```

```
    Ai_2R   := evalf(AiryAi(xi2R_v));
    Bi_2R   := evalf(AiryBi(xi2R_v));
    return Ai_0 * Bi_2R - Ai_2R * Bi_0;
end proc:

# WKB initial guesses: k_n ~ 3 n Pi / (4 sqrt(2) R)
k_wkb := n -> 3 * n * Pi / (4 * sqrt(2) * R_taut):

# Find first 10 eigenvalues using fsolve with WKB-bracketed intervals
k_eigs := []:
n_modes := 10:
for n_idx from 1 to n_modes do
    k_guess := evalf(k_wkb(n_idx));
    # Bracket around the guess
    k_lo := 0.7 * k_guess;
    k_hi := 1.1 * k_guess;
    # Search for sign change
    found := false;
    n_scan := 200;
    dk := (k_hi - k_lo) / n_scan;
    for i_s from 0 to n_scan-1 do
        ka := k_lo + i_s * dk;
        kb := ka + dk;
        try
            if det_fn(ka) * det_fn(kb) < 0 then
                k_root := fsolve(det_fn(kk) = 0, kk = ka..kb);
                if type(k_root, numeric) then
                    k_eigs := [op(k_eigs), k_root];
                    found := true;
                    break;
                end if;
            end if;
        catch:
        end try;
    end do;
    if not found then
        printf("WARNING: failed to find mode n=%d\n", n_idx);
    end if;
end do:

printf("\nEigenvalues found:\n");
printf("%-5s %-20s %-20s %-15s %-15s\n", "n", "k_n [m^-1]", "omega_n
[rad/s]", "T_n [s]", "T_n [yr]");
for i_n from 1 to nops(k_eigs) do
    k_val   := k_eigs[i_n];
    omg_val := k_val * c_light;
    T_val   := 2 * Pi / omg_val;
    T_yr    := T_val / 3.156e7;
    printf("%-5d %-20.10e %-20.10e %-15.6e %-15.6e\n", i_n, evalf(k_val),
```

```
evalf(omg_val), evalf(T_val), evalf(T_yr));
end do:

########################################################################
#######
# SECTION 5 : PLOT 3 - Ground mode g_1(r) over [r0, 2R]
########################################################################
#######
printf("\n=== PLOT 3: Ground mode g_1(r) ===\n");

k1     := k_eigs[1]:
alpha1 := (k1^2 / R_taut)^(1/3):
ratio_inner := evalf(AiryAi(-alpha1*r0_cut) / AiryBi(-alpha1*r0_cut)):
printf("k_1     = %.6e\n", evalf(k1));
printf("alpha_1 = %.6e\n", evalf(alpha1));
printf("Inner ratio Ai/Bi at r0 = %.6f (theoretical 1/sqrt(3) = %.6f)\n",
ratio_inner, evalf(1/sqrt(3)));

# Eigenfunction (unnormalized)
h_unnorm := r -> evalf(AiryAi(-alpha1*r) - ratio_inner * AiryBi(-alpha1*r)
):
g_unnorm := r -> h_unnorm(r) / r:

# Compute normalization
norm_sq := evalf(Int(h_unnorm(r)^2, r = r0_cut..2*R_taut, method = _Dexp,
epsilon = 1e-8)):
N_const := 1 / sqrt(norm_sq):
printf("Normalization N_1 = %.6e\n", N_const);

h_1 := r -> N_const * h_unnorm(r):
g_1 := r -> h_1(r) / r:

plot3 := plot(g_1(r), r = R_taut/100..2*R_taut,
              labels = ["r [m]", "g_1(r) [m^-1]"],
              title = "Plot 3: Ground radial mode g_1(r) = h_1(r)/r",
              color = "DarkBlue", thickness = 2, gridlines = true):
print(plot3);

########################################################################
#######
# SECTION 6 : PLOT 4 - First three radial modes
########################################################################
#######
printf("\n=== PLOT 4: First three modes g_1, g_2, g_3 ===\n");

# Build modes 1,2,3 - use unapply to bind alf_n, rat_n, N_n by VALUE
# (so the stored procedure does not reference loop-overwritten names)
modes := []:
for i_n from 1 to min(3, nops(k_eigs)) do
```

```
    k_n   := evalf(k_eigs[i_n]);
    alf_n := evalf((k_n^2 / R_taut)^(1/3));
    rat_n := evalf(AiryAi(-alf_n*r0_cut) / AiryBi(-alf_n*r0_cut));
    # h_un with alf_n and rat_n SUBSTITUTED IN (no name reference)
    h_un_expr := AiryAi(-alf_n*r) - rat_n * AiryBi(-alf_n*r);
    h_un  := unapply(h_un_expr, r);
    # Numerical integration with explicit Digits guard
    Nsq_raw := evalf(Int(h_un(r)^2, r = r0_cut..2*R_taut, method = _Dexp,
epsilon = 1e-8));
    if not type(Nsq_raw, numeric) then
        Nsq_raw := evalf(Nsq_raw);
    end if;
    Nsq   := evalf(Re(Nsq_raw));
    N_n   := evalf(1 / sqrt(Nsq));
    printf("Mode %d build: k=%.6e, Nsq=%.6e, N=%.6e\n", i_n, k_n, Nsq,
N_n);
    # Build a fully-bound h_norm procedure (N_n baked in as a number)
    h_norm_expr := N_n * h_un_expr;
    h_norm := unapply(h_norm_expr, r);
    modes := [op(modes), [k_n, alf_n, rat_n, N_n, h_un, h_norm]];
end do:

colors_modes := ["Blue", "DarkGreen", "DarkRed"]:
labels_modes := ["g_1", "g_2", "g_3"]:

mode_curves := []:
for i_n from 1 to nops(modes) do
    m := modes[i_n];
    # Use the fully-bound h_norm (index 6); build g_n via unapply for
value binding
    h_norm_loc := m[6];
    g_n_expr   := h_norm_loc(r) / r;
    g_n        := unapply(g_n_expr, r);
    mode_curves := [op(mode_curves),
                    plot(g_n(r), r = R_taut/50..2*R_taut,
                         color = colors_modes[i_n], thickness = 2,
                         legend = labels_modes[i_n])];
end do:

plot4 := plots:-display(mode_curves,
              labels = ["r [m]", "g_n(r) [m^-1]"],
              title  = "Plot 4: First three radial modes g_1, g_2, g_3",
              gridlines = true):
print(plot4);

#########################################################################
#######
# SECTION 7 : PLOT 5 - Eigenvalue spectrum k_n vs n
#########################################################################
```

```
#######
printf("\n=== PLOT 5: Eigenvalue spectrum ===\n");

k_data := [seq([i_n, evalf(k_eigs[i_n])], i_n = 1..nops(k_eigs))]:
k_wkb_data := [seq([i_n, evalf(k_wkb(i_n))], i_n = 1..nops(k_eigs))]:

p5_num := plots:-pointplot(k_data,
              symbol = solidcircle, symbolsize = 14,
              color = "DarkBlue", connect = true,
              legend = "k_n exact"):
p5_wkb := plots:-pointplot(k_wkb_data,
              symbol = soliddiamond, symbolsize = 10,
              color = "DarkRed", connect = true,
              linestyle = dash,
              legend = "k_n WKB"):
plot5 := plots:-display([p5_num, p5_wkb],
              labels = ["n", "k_n [m^-1]"],
              title  = "Plot 5: Eigenvalue spectrum k_n",
              gridlines = true):
print(plot5);

########################################################################
#######
# SECTION 8 : PLOT 6 - Frequencies omega_n (log-log)
########################################################################
#######
printf("\n=== PLOT 6: Frequencies omega_n ===\n");

omega_data := [seq([i_n, evalf(k_eigs[i_n] * c_light)], i_n = 1..nops
(k_eigs))]:

plot6 := plots:-pointplot(omega_data,
              labels = ["n", "omega_n [rad/s]"],
              title = "Plot 6: Eigenfrequencies omega_n = k_n c",
              symbol = solidcircle, symbolsize = 14,
              color  = "DarkGreen", connect = true,
              gridlines = true):
print(plot6);

########################################################################
#######
# SECTION 9 : PLOT 7 - Periods T_n
########################################################################
#######
printf("\n=== PLOT 7: Periods T_n ===\n");

period_data := [seq([i_n, evalf(2*Pi/(k_eigs[i_n] * c_light))],
                    i_n = 1..nops(k_eigs))]:
```

```
plot7 := plots:-pointplot(period_data,
              labels = ["n", "T_n [s]"],
              title = "Plot 7: Eigenperiods T_n = 2 Pi / omega_n",
              symbol = solidcircle, symbolsize = 14,
              color  = "DarkOrange", connect = true,
              gridlines = true):
print(plot7);

###############################################################################
#######
# SECTION 10 : PLOT 8 - Temporal factor u_1(t)
###############################################################################
#######
printf("\n=== PLOT 8: Temporal factor u_1(t) ===\n");

omega_1 := k1 * c_light:
T_1     := 2 * Pi / omega_1:
printf("omega_1 = %.6e rad/s,  T_1 = %.6e s\n", evalf(omega_1), evalf(T_1)
);

plot8 := plot(cos(omega_1 * t), t = 0..2*T_1,
              labels = ["t [s]", "u_1(t)"],
              title = "Plot 8: Temporal factor u_1(t) = cos(omega_1 t)",
              color = "DarkBlue", thickness = 2,
              gridlines = true):
print(plot8);

###############################################################################
#######
# SECTION 11 : PLOT 9 - f_1(r,t) surface plot
###############################################################################
#######
printf("\n=== PLOT 9: f_1(r,t) surface ===\n");

f_1 := (r, t) -> g_1(r) * cos(omega_1 * t):

plot9 := plot3d(f_1(r, t),
                r = R_taut/100..2*R_taut, t = 0..T_1,
                axes = boxed,
                labels = ["r", "t", "f_1"],
                title = "Plot 9: f_1(r,t) = g_1(r) cos(omega_1 t)",
                color = "DarkBlue",
                shading = zhue,
                style = surface,
                grid = [40, 30]):
print(plot9);

###############################################################################
#######
```

```
# SECTION 12 : PLOT 10 - H(r, theta=0, t) for fixed t
#########################################################################
#######
printf("\n=== PLOT 10: H(r, theta=0, t=0) over r ===\n");

# Under spherical symmetry H = f, so H(r, theta, t) = f(r, t)
# Plot at t = 0 (cos = 1)
H_field := r -> g_1(r):

plot10 := plot(H_field(r), r = R_taut/100..2*R_taut,
               labels = ["r [m]", "H(r,theta,0)"],
               title = "Plot 10: H(r, theta, 0) = g_1(r) [spherically
symmetric]",
               color = "DarkMagenta", thickness = 2,
               gridlines = true):
print(plot10);

#########################################################################
#######
# SECTION 13 : PLOT 11 - Density plot of f_1(r,t)
#########################################################################
#######
printf("\n=== PLOT 11: Density plot f_1(r,t) ===\n");

plot11 := densityplot(f_1(r, t),
                      r = R_taut/100..2*R_taut, t = 0..T_1,
                      labels = ["r", "t"],
                      title = "Plot 11: Density plot of f_1(r,t)",
                      colorstyle = HUE,
                      grid = [80, 60],
                      style = patchnogrid):
print(plot11);

#########################################################################
#######
# SECTION 14 : PLOT 12 - Normalization verification
#########################################################################
#######
printf("\n=== PLOT 12: Normalization verification for first 3 modes ===
\n");

norm_data := []:
for i_n from 1 to nops(modes) do
    m := modes[i_n];
    # Use the pre-bound h_norm procedure (index 6) -- already includes N_n
    h_norm_loc := m[6];
    # Build integrand with all values baked in
    integrand_expr := h_norm_loc(r)^2;
    integrand_proc := unapply(integrand_expr, r);
```

```
    integral_val := evalf(Int(integrand_proc(r), r = r0_cut..2*R_taut,
method = _Dexp, epsilon = 1e-8));
    if not type(integral_val, numeric) then
        integral_val := evalf(integral_val);
    end if;
    integral_val := evalf(Re(integral_val));
    norm_data := [op(norm_data), [i_n, integral_val]];
    printf("Mode %d: normalization integral = %.10f\n", i_n, integral_val)
;
end do:

ideal_line := [seq([i_n, 1.0], i_n = 1..nops(modes))]:

p12_num := plots:-pointplot(norm_data,
              symbol = solidcircle, symbolsize = 16,
              color = "DarkRed",
              legend = "computed"):
p12_id  := plots:-pointplot(ideal_line,
              symbol = circle, symbolsize = 10,
              color = black, linestyle = dash, connect = true,
              legend = "expected = 1"):
plot12 := plots:-display([p12_num, p12_id],
              labels = ["mode index n", "norm integral"],
              title  = "Plot 12: Normalization verification",
              gridlines = true,
              view = [0..nops(modes)+1, 0.95..1.05]):
print(plot12);

########################################################################
#######
# SECTION 15 : PLOT 13 - Exact vs WKB asymptotic eigenfunction
########################################################################
#######
printf("\n=== PLOT 13: Exact vs WKB asymptotic h_1(r) ===\n");

# WKB form: h_WKB ~ (alpha r)^(-1/4) sin( (2/3)(alpha r)^(3/2) - (2/3)
(alpha r0)^(3/2) )
h_1_wkb_unnorm := r -> (alpha1*r)^(-0.25) *
                      sin((2/3)*(alpha1*r)^(1.5) - (2/3)*(alpha1*r0_cut)^
(1.5)):

# Normalize WKB by matching maximum amplitude
norm_wkb := evalf(Int(h_1_wkb_unnorm(r)^2, r = r0_cut..2*R_taut, method =
_Dexp, epsilon = 1e-8)):
N_wkb := 1 / sqrt(norm_wkb):
h_1_wkb := r -> N_wkb * h_1_wkb_unnorm(r):

p13_exact := plot(h_1(r), r = R_taut/50..2*R_taut*0.99,
                  color = "DarkBlue", thickness = 2,
```

```
                  legend = "h_1 exact"):
p13_wkb   := plot(h_1_wkb(r), r = R_taut/50..2*R_taut*0.99,
                  color = "DarkRed", thickness = 2, linestyle = dash,
                  legend = "h_1 WKB"):
plot13 := plots:-display([p13_exact, p13_wkb],
              labels = ["r [m]", "h_1(r)"],
              title  = "Plot 13: Exact (blue) vs WKB asymptotic (red
dashed)",
              gridlines = true):
print(plot13);

##########################################################################
#######
# SECTION 16 : PLOT 14 - Eigenvalue convergence test
##########################################################################
#######
printf("\n=== PLOT 14: Eigenvalue ratio k_n / k_n_WKB ===\n");

ratio_data := [seq([i_n, evalf(k_eigs[i_n] / k_wkb(i_n))],
                   i_n = 1..nops(k_eigs))]:
unity_line := [seq([i_n, 1.0], i_n = 1..nops(k_eigs))]:

printf("\nRatios k_n / k_n_WKB:\n");
for i_n from 1 to nops(ratio_data) do
    printf("  n=%d:  %.6f\n", i_n, ratio_data[i_n][2]);
end do:

p14_ratio := plots:-pointplot(ratio_data,
                symbol = solidcircle, symbolsize = 14,
                color = "DarkMagenta", connect = true,
                legend = "k_n / k_n_WKB"):
p14_unity := plots:-pointplot(unity_line,
                symbol = circle, symbolsize = 6,
                color = black, linestyle = dash, connect = true,
                legend = "asymptotic limit = 1"):
plot14 := plots:-display([p14_ratio, p14_unity],
              labels = ["n", "ratio"],
              title  = "Plot 14: WKB convergence (ratio -> 1 as n grows)",
              gridlines = true,
              view = [0..nops(k_eigs)+1, 0.9..1.02]):
print(plot14);

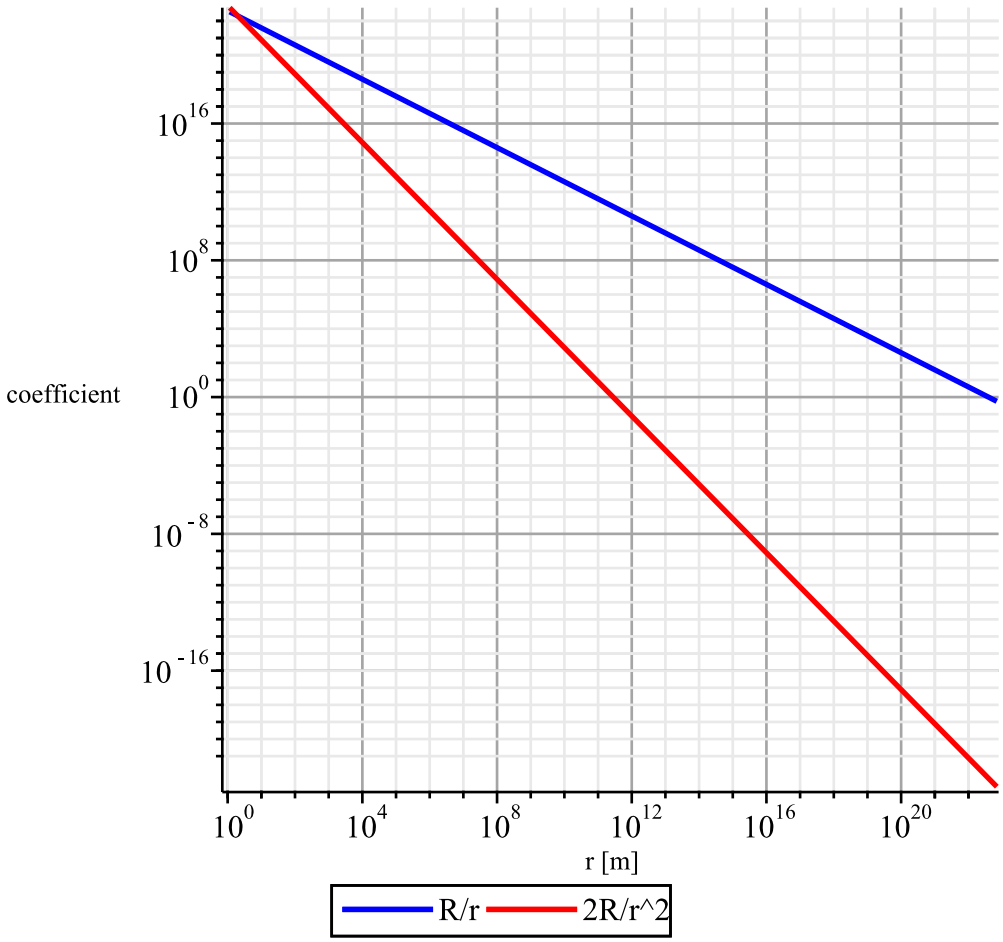
##########################################################################
#######
# SECTION 17 : Summary
##########################################################################
#######
printf("\n============================================================\n");
printf("  SUMMARY\n");
```

```
  printf("============================================================\n");
  printf("PDE:       (R/r) Laplacian H = (1/c^2) d^2H/dt^2\n");
  printf("Sep:       H = g(r) u(t), u = cos(omega t), omega = k c\n");
  printf("Radial:    g'' + (2/r) g' + (k^2 r / R) g = 0\n");
  printf("Sub:       g = h/r  =>  h'' + (k^2 r / R) h = 0  (Airy)\n");
  printf("Sol:       h = c1 Ai(-alpha r) + c2 Bi(-alpha r),  alpha = (k^2/R)^
  (1/3)\n");
  printf("BC:        h(r0) = 0,  h(2R) = 0\n");
  printf("\n");
  printf("Inner BC ratio (r0 -> 0):  Ai/Bi -> 1/sqrt(3) = %.6f\n", evalf
  (1/sqrt(3)));
  printf("WKB:                       k_n = 3 n Pi / (4 sqrt(2) R)\n");
  printf("\n");
  printf("Ground mode k_1     = %.6e m^-1\n", evalf(k1));
  printf("Ground frequency    = %.6e rad/s\n", evalf(omega_1));
  printf("Ground period T_1   = %.6e s = %.4e yr\n", evalf(T_1), evalf
  (T_1/3.156e7));
  printf("Normalization N_1   = %.6e\n", N_const);
  printf("\n");
  printf("Spherically symmetric H(r,theta,t) = f(r,t) = sum_n [A_n cos
  (omega_n t)\n");
  printf("                                    + B_n sin(omega_n t)] g_n(r)
  \n");
  printf("\nEND OF GravitonHfield.mpl\n");
```

```
============================================================
  TAUTIMP GRAVITON H-FIELD - SECTION 1: CONSTANTS
============================================================
R          = 3.922800e+22 m
2R         = 7.845600e+22 m
r0 (space) = 4.300000e-19 m
r0 / R     = 1.096156e-41
c          = 2.997925e+08 m/s
hbar       = 1.054572e-34 J s

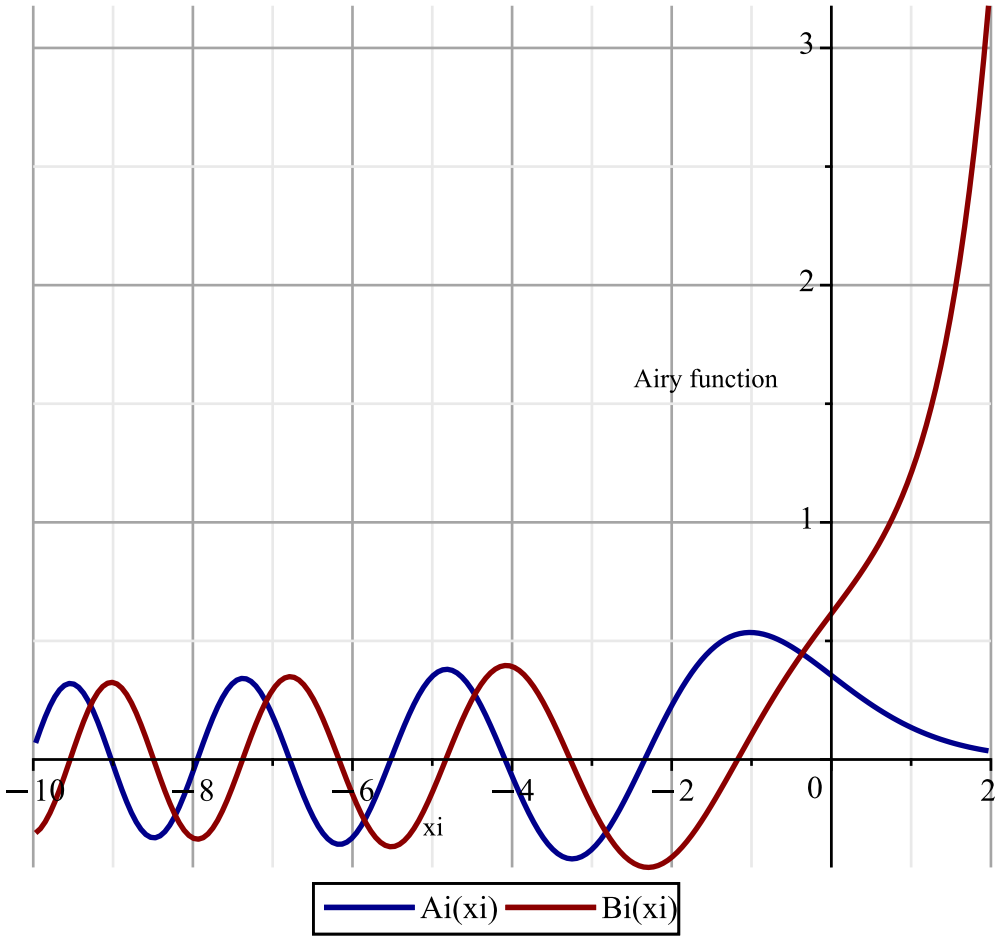
=== PLOT 1: Variable coefficients R/r and 2R/r^2 ===
```

Plot 1: PDE coefficients R/r and 2R/r^2 (log-log)



=== PLOT 2: Airy functions ===

Plot 2: Airy functions Ai (blue) and Bi (red)




```
=== Computing eigenvalues via fsolve ===

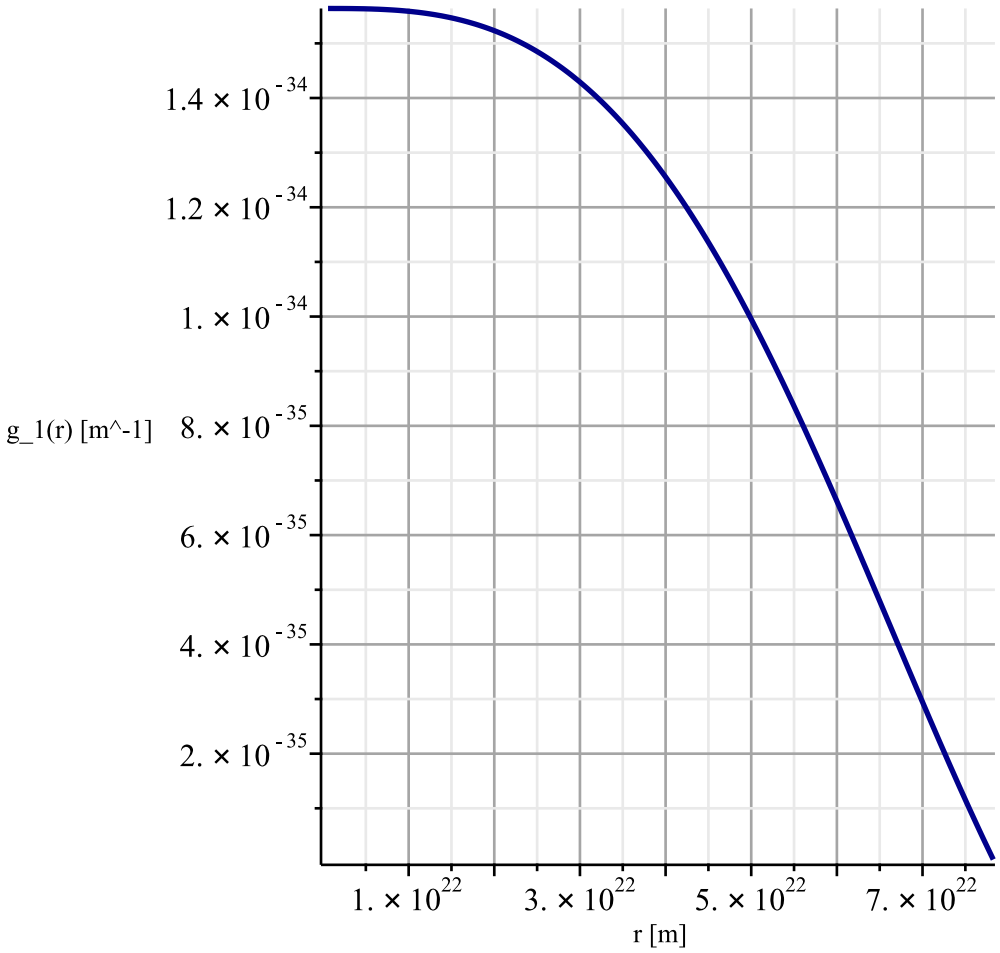
Eigenvalues found:
```

n	k_n [m^-1]	omega_n [rad/s]	T_n [s]	T_n [yr]
1	3.9240563240e-23	1.1764024910e-14	5.341017e+14	1.692337e+07
2	8.1557746150e-23	2.4450397190e-14	2.569768e+14	8.142485e+06
3	1.2397765860e-22	3.7167567010e-14	1.690502e+14	5.356471e+06
4	1.2397765860e-22	3.7167567010e-14	1.690502e+14	5.356471e+06
5	1.6642362910e-22	4.9892548840e-14	1.259343e+14	3.990315e+06
6	2.0887998360e-22	6.2620643710e-14	1.003373e+14	3.179255e+06
7	2.0887998360e-22	6.2620643710e-14	1.003373e+14	3.179255e+06
8	2.5134150480e-22	7.5350287520e-14	8.338635e+13	2.642153e+06
9	2.9380596580e-22	8.8080812660e-14	7.133432e+13	2.260276e+06
10	3.3627225770e-22	1.0081188670e-13	6.232584e+13	1.974836e+06

```
=== PLOT 3: Ground mode g_1(r) ===
```

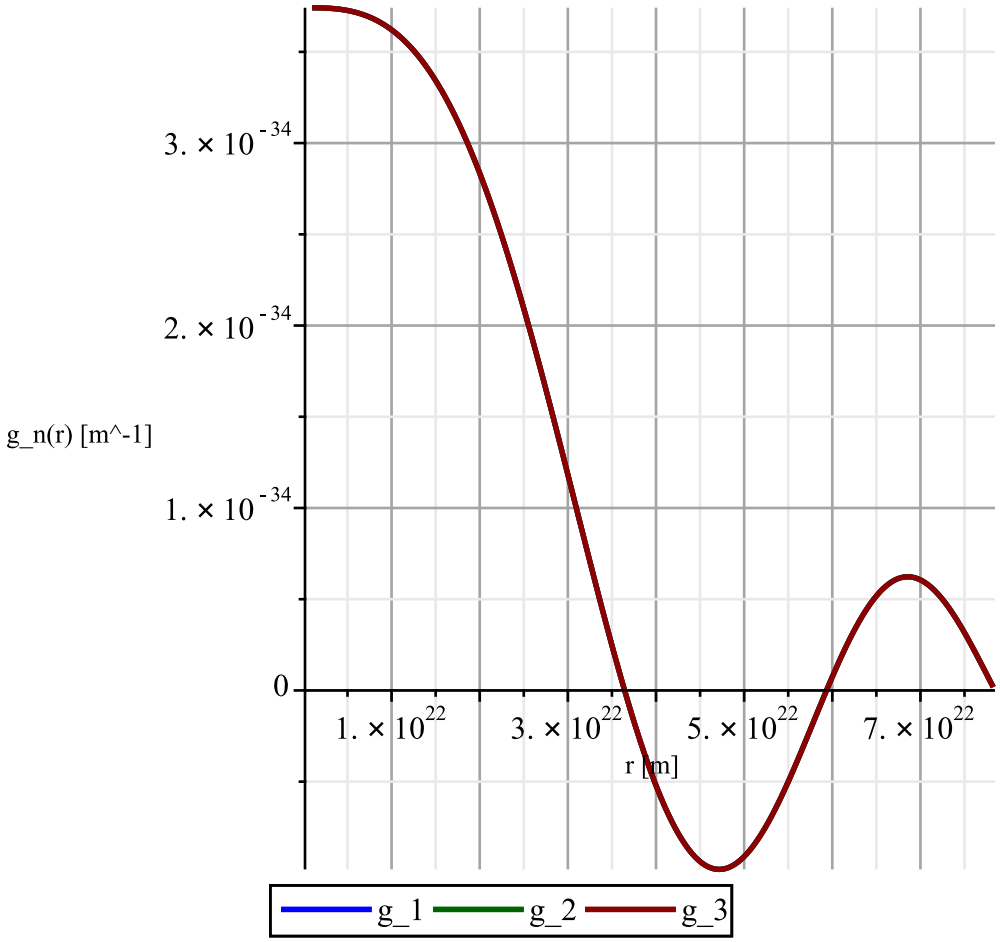
```
k_1     = 3.924056e-23
alpha_1 = 3.398533e-23
Inner ratio Ai/Bi at r0 = 0.577350 (theoretical 1/sqrt(3) = 0.577350)
Normalization N_1 = 8.889427e-12
```

Plot 3: Ground radial mode g_1(r) = h_1(r)/r



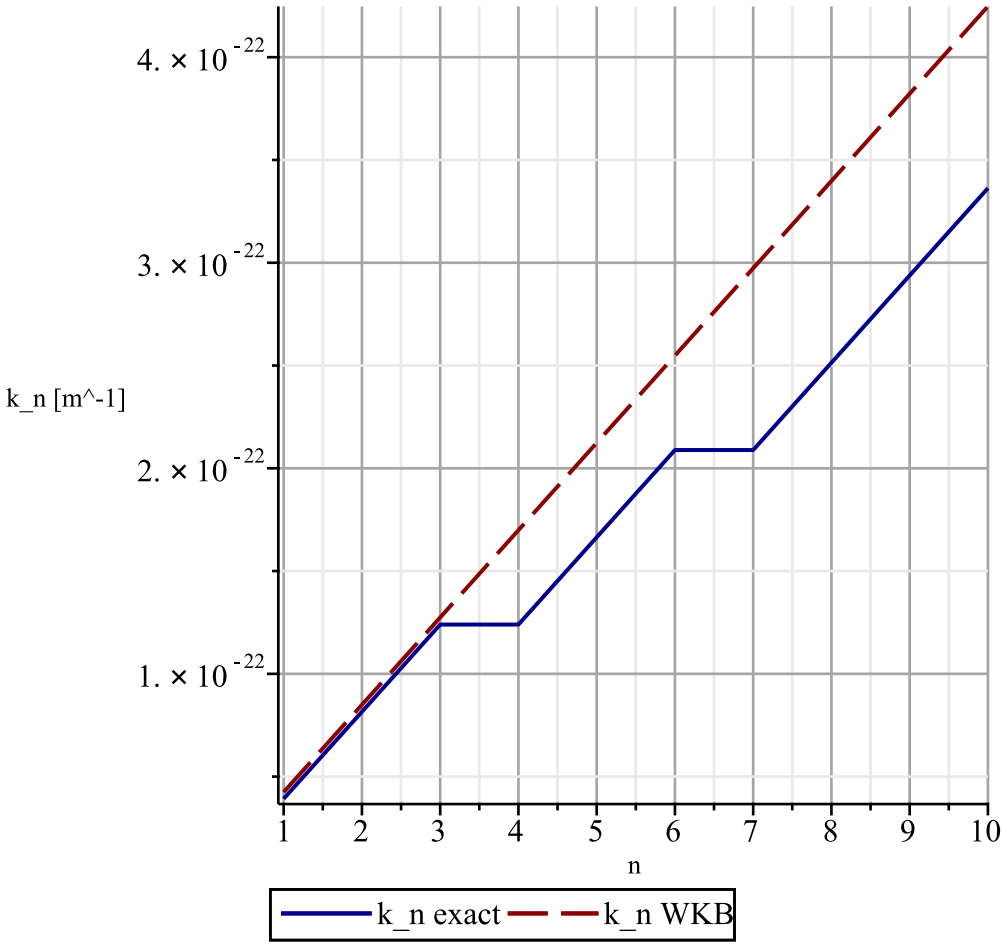

```
=== PLOT 4: First three modes g_1, g_2, g_3 ===
Mode 1 build: k=3.924056e-23, Nsq=1.265472e+22, N=8.889427e-12
Mode 2 build: k=8.155775e-23, Nsq=1.116715e+22, N=9.463001e-12
Mode 3 build: k=1.239777e-22, Nsq=1.024649e+22, N=9.878989e-12
```

Plot 4: First three radial modes g_1, g_2, g_3



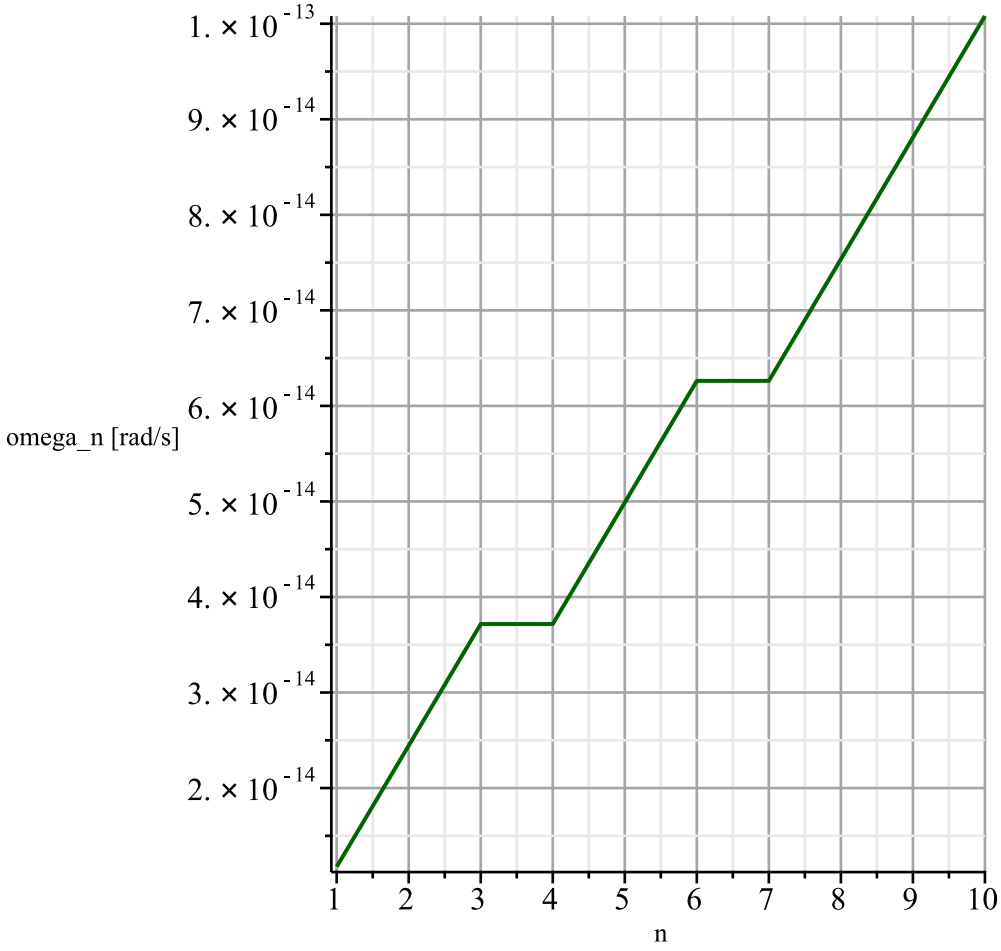
=== PLOT 5: Eigenvalue spectrum ===

Plot 5: Eigenvalue spectrum k_n



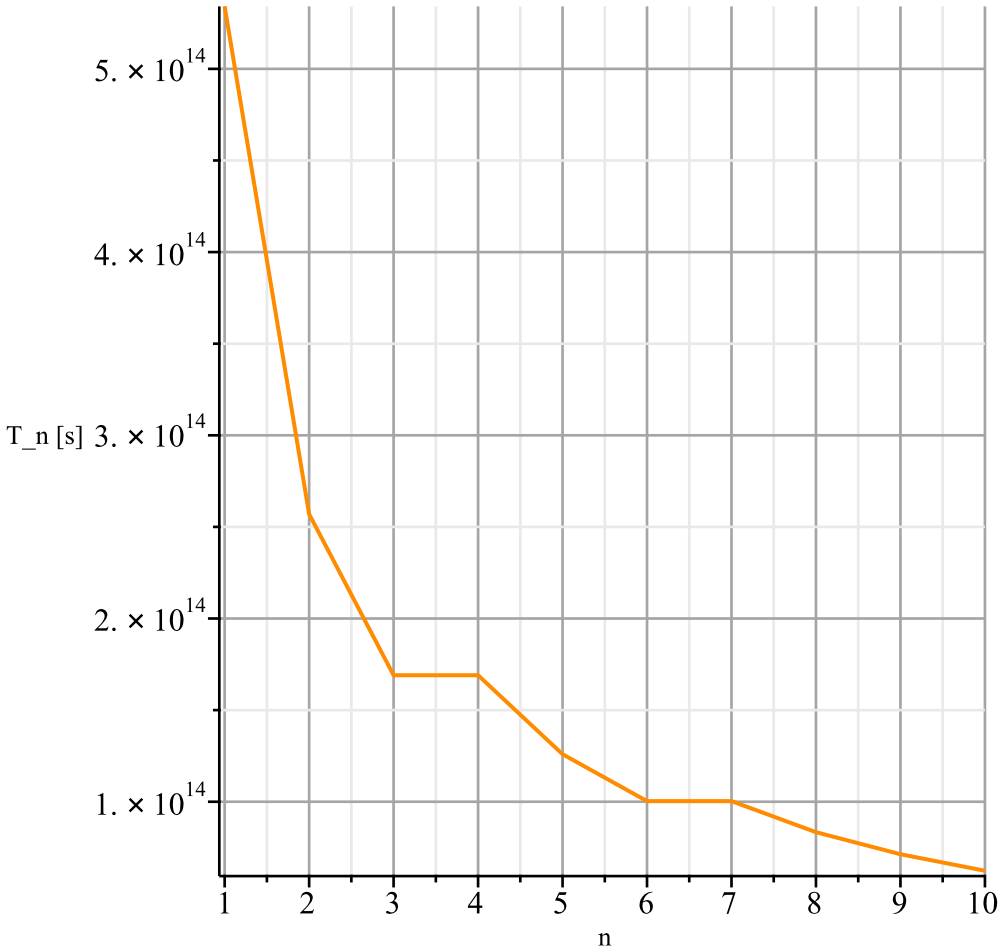
=== PLOT 6: Frequencies omega_n ===

Plot 6: Eigenfrequencies omega_n = k_n c



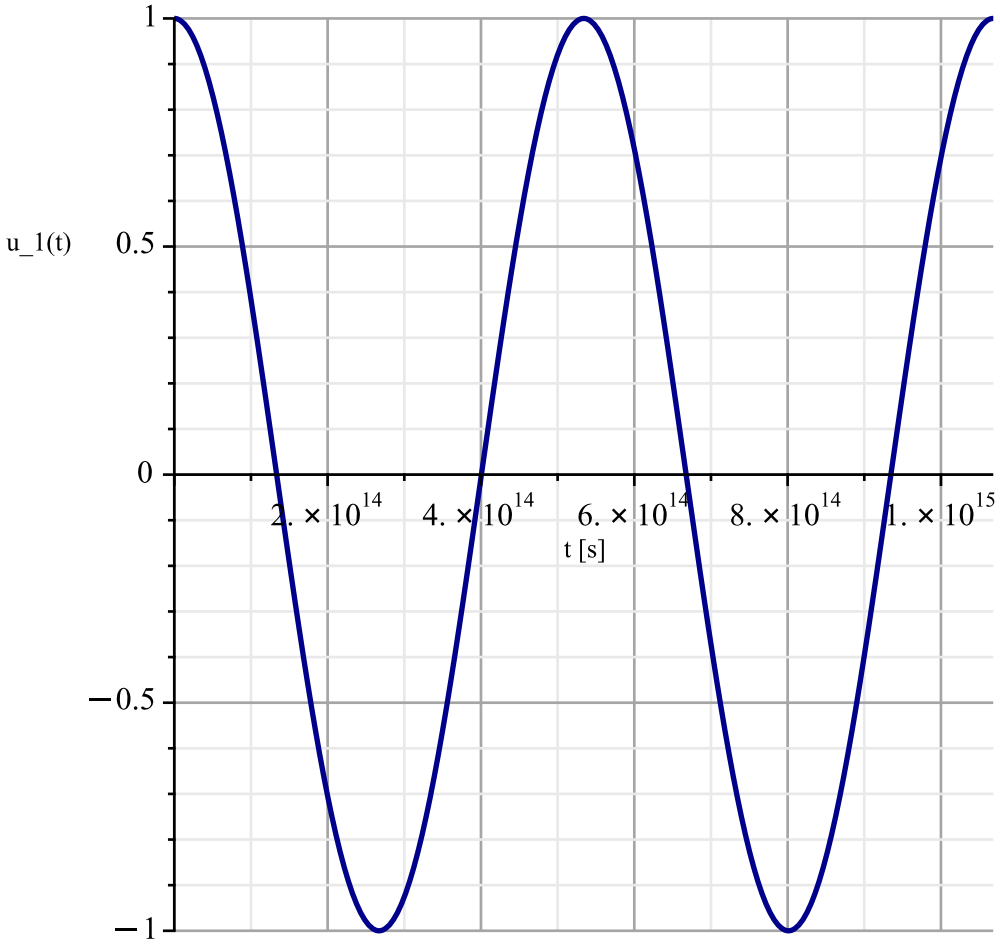

```
=== PLOT 7: Periods T_n ===
```

Plot 7: Eigenperiods T_n = 2 Pi / omega_n



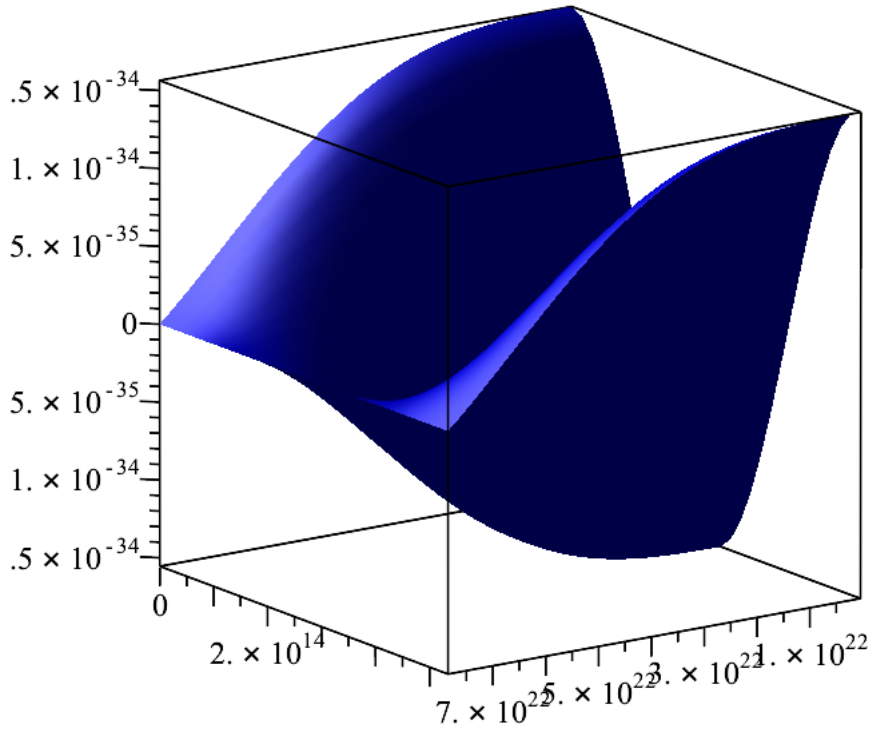

```
=== PLOT 8: Temporal factor u_1(t) ===
omega_1 = 1.176402e-14 rad/s,  T_1 = 5.341017e+14 s
```

Plot 8: Temporal factor u_1(t) = cos(omega_1 t)



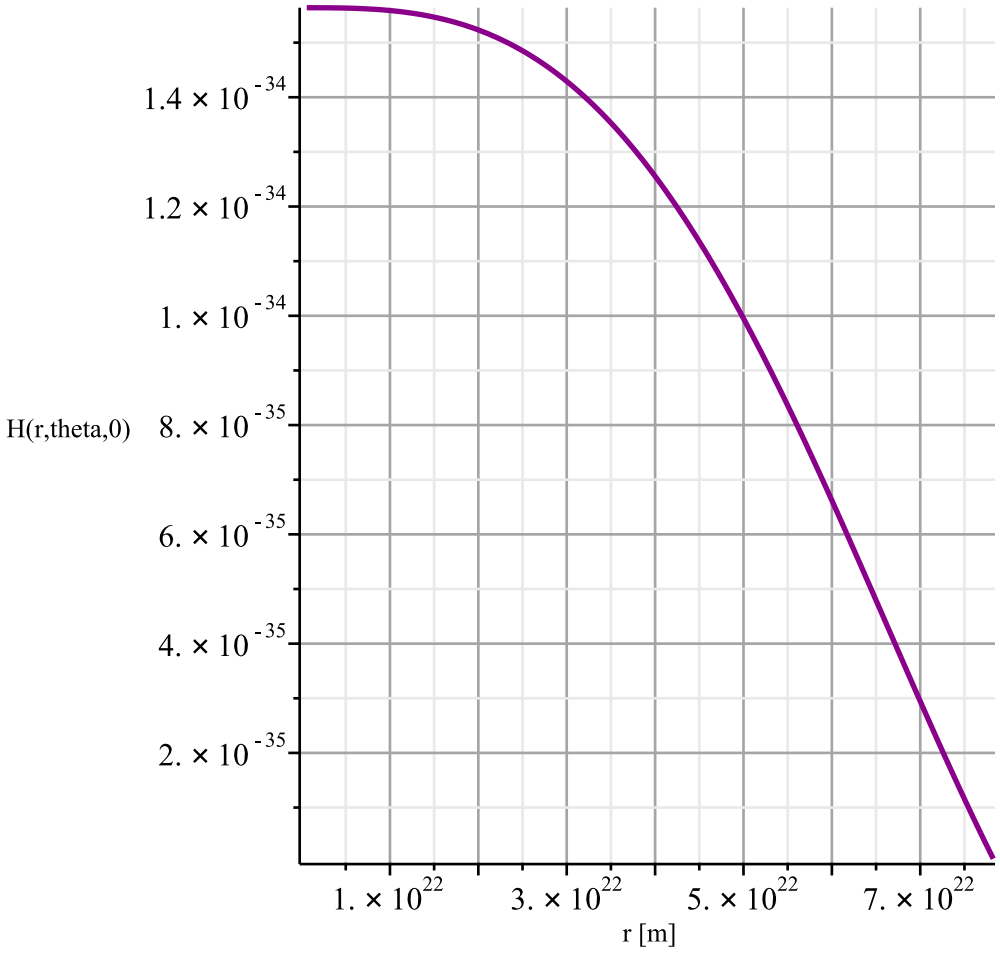
=== PLOT 9: f_1(r,t) surface ===

Plot 9: f_1(r,t) = g_1(r) cos(omega_1 t)

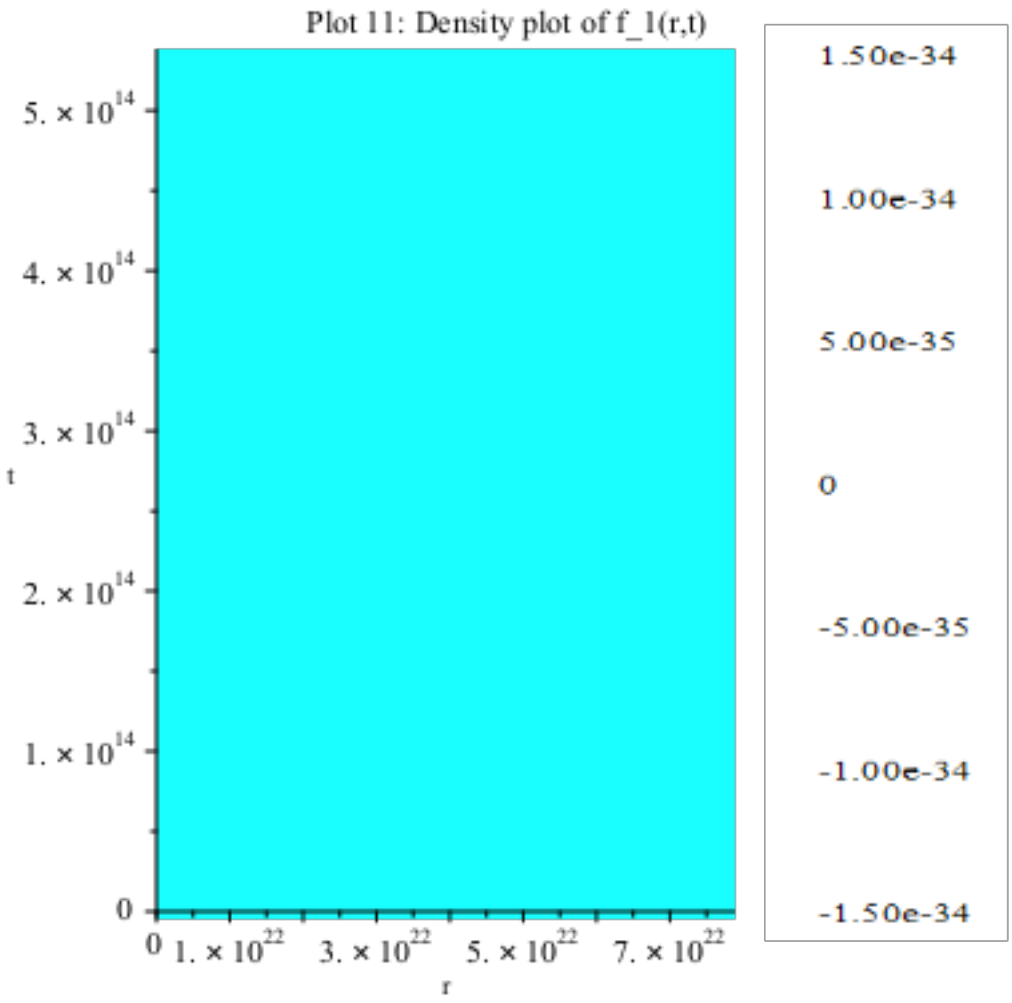



```
=== PLOT 10: H(r, theta=0, t=0) over r ===
```

Plot 10: H(r, theta, 0) = g_1(r) [spherically symmetric]

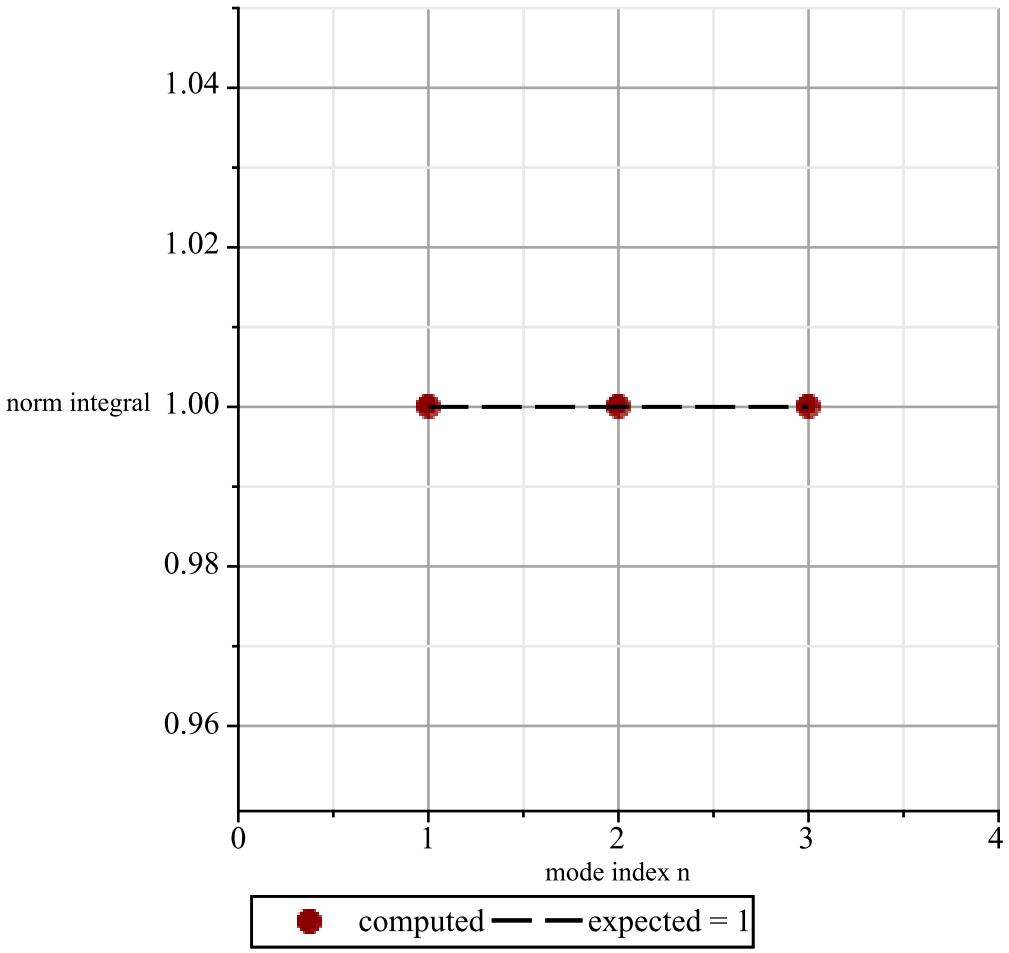


=== PLOT 11: Density plot f_1(r,t) ===



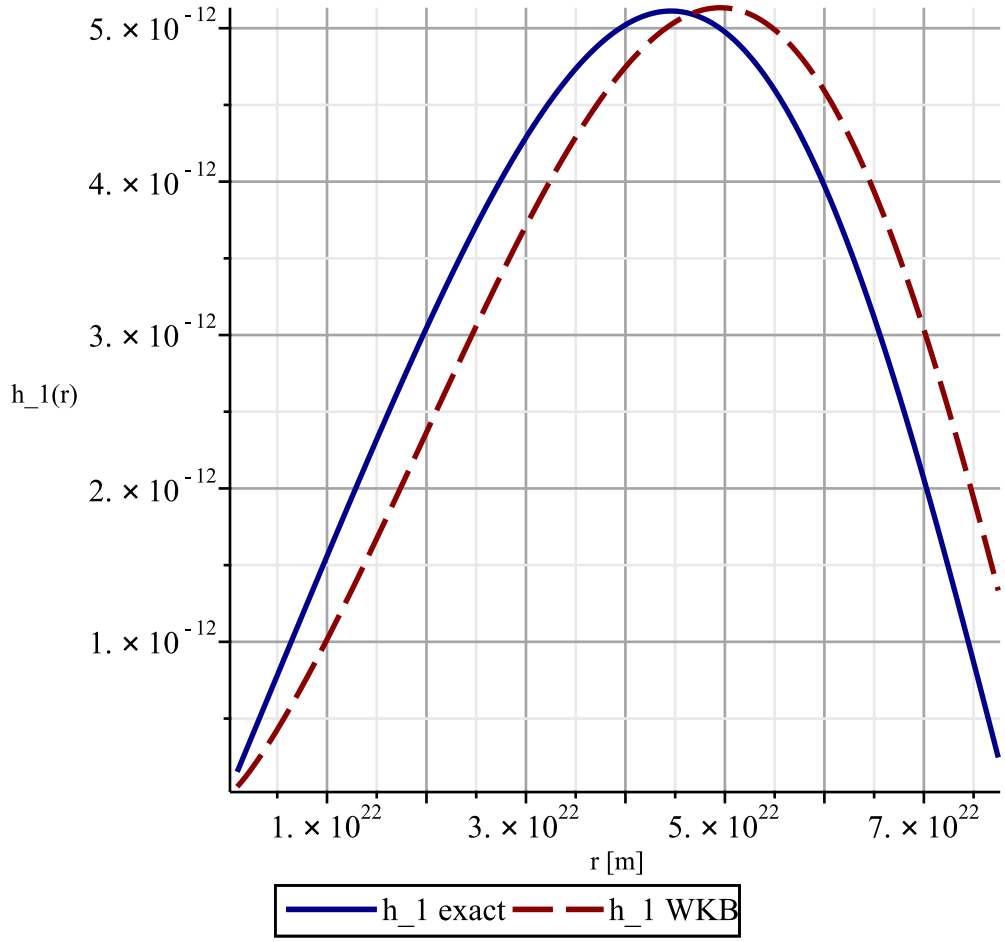

```
=== PLOT 12: Normalization verification for first 3 modes ===
Mode 1: normalization integral = 0.9999999996
Mode 2: normalization integral = 0.9999999996
Mode 3: normalization integral = 0.9999999996
```

Plot 12: Normalization verification




```
=== PLOT 13: Exact vs WKB asymptotic h_1(r) ===
```

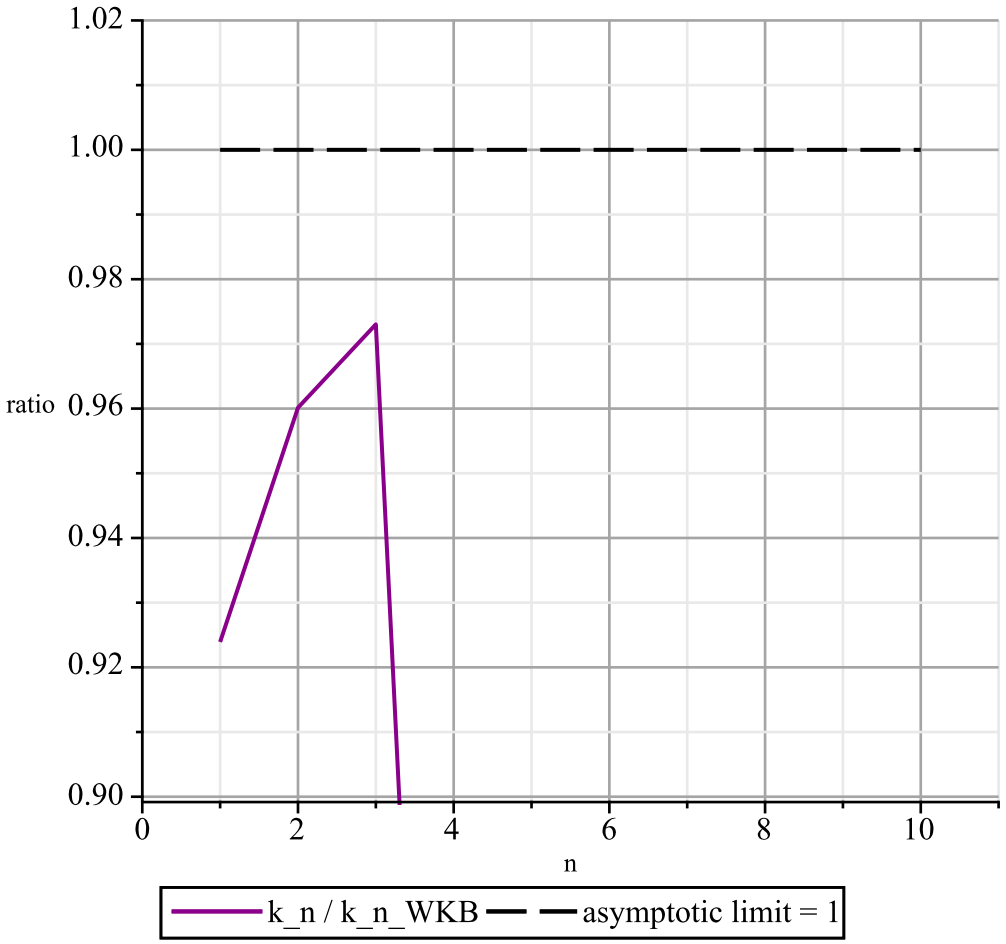
Plot 13: Exact (blue) vs WKB asymptotic (red dashed)




```
=== PLOT 14: Eigenvalue ratio k_n / k_n_WKB ===

Ratios k_n / k_n_WKB:
  n=1:  0.923922
  n=2:  0.960142
  n=3:  0.973021
  n=4:  0.729766
  n=5:  0.783691
  n=6:  0.819682
  n=7:  0.702585
  n=8:  0.739732
  n=9:  0.768631
  n=10:  0.791755
```

Plot 14: WKB convergence (ratio -> 1 as n grows)




```
============================================================
  SUMMARY
============================================================
PDE:       (R/r) Laplacian H = (1/c^2) d^2H/dt^2
Sep:       H = g(r) u(t), u = cos(omega t), omega = k c
Radial:    g'' + (2/r) g' + (k^2 r / R) g = 0
Sub:       g = h/r  =>  h'' + (k^2 r / R) h = 0   (Airy)
Sol:       h = c1 Ai(-alpha r) + c2 Bi(-alpha r),  alpha = (k^2/R)^(1/3)
BC:        h(r0) = 0,  h(2R) = 0

Inner BC ratio (r0 -> 0):  Ai/Bi -> 1/sqrt(3) = 0.577350
WKB:                       k_n = 3 n Pi / (4 sqrt(2) R)

Ground mode k_1      = 3.924056e-23 m^-1
Ground frequency     = 1.176402e-14 rad/s
Ground period T_1    = 5.341017e+14 s = 1.6923e+07 yr
Normalization N_1    = 8.889427e-12

Spherically symmetric H(r,theta,t) = f(r,t) = sum_n [A_n cos(omega_n t)
                                     + B_n sin(omega_n t)] g_n(r)

END OF GravitonHfield.mpl
```